实验报告

课程名称：Design pattern and software architecture

学　　院：计算机科学与工程学院

专　　业:Computer Science and Technology　班　　级：SE-1

姓　　名：Muhammad Samudera　学　　号：

年　　月　　日

山东科技大学教务处制

实 验 报 告

组　别	1	姓　名	Samudera	同组实验者	None
实验项目名称	Singleton Pattern				
教师评语	Good understanding of the pattern idea and implementation. The report is clear and written in proper English.				
实验成绩:		指导教师（签名）： 2026 3 23			

1. Objective

The objective of this experiment is to implement the Singleton design pattern using a database connection management example involving DatabaseConfig and ConnectionPool. The goal of this pattern is to ensure that a class has only one instance throughout the application's lifecycle, providing a global point of access to that instance.

In this example, the program has two Singleton classes:

- DatabaseConfig : stores database configuration such as host, port, credentials, and pool size.
- ConnectionPool : manages a pool of active database connections, relying on DatabaseConfig to read its settings.

Each Singleton is responsible for controlling its own instantiation. The Application class acts as the client that retrieves and uses these Singletons through the static getInstance() method.

2. Principle

The Singleton pattern restricts the instantiation of a class to a single object. Instead of allowing clients to create instances directly using new, the class itself controls the creation through a private constructor and a public static accessor method.

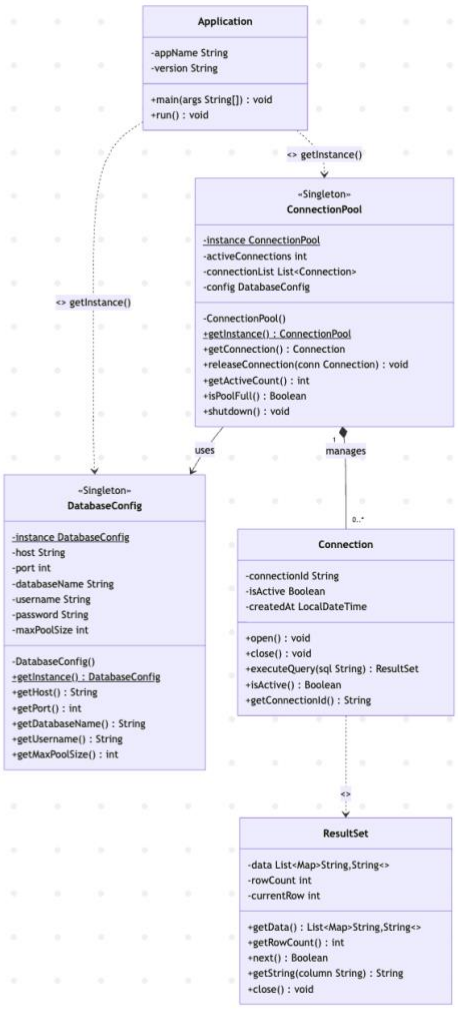
Key benefits of this separation include:

- Only one instance of DatabaseConfig and ConnectionPool exists at any time, preventing conflicting configurations.
- Global access to the single instance is provided via getInstance() without passing objects around.
- Resources such as database connections are efficiently managed and shared across the

application.

- Thread-safety is guaranteed through the synchronized keyword on getInstance(), preventing race conditions in multi-threaded environments.

3. UML Diagram



4. Key Code (Java)

```
class DatabaseConfig {

    private static DatabaseConfig instance;                    // single instance holder
```

```
    private DatabaseConfig() {                                  // private constructor
        this.host = "localhost";    this.port = 5432;
        this.databaseName = "app_db";    this.username = "admin";
        this.maxPoolSize = 10;
    }

    public static synchronized DatabaseConfig getInstance() {
        if (instance == null) { instance = new DatabaseConfig(); }
        return instance;
    }
}

class ConnectionPool {
    private static ConnectionPool instance;                     // single instance holder
    private final DatabaseConfig config;

    private ConnectionPool() {                                  // private constructor
        this.config = DatabaseConfig.getInstance();             // reuses existing singleton
        this.connectionList = new ArrayList<>();
        this.activeConnections = 0;
    }
```

```
    public static synchronized ConnectionPool getInstance() {
        if (instance == null) { instance = new ConnectionPool(); }
        return instance;
    }

    public Connection getConnection() {
        if (isPoolFull()) return null;
        Connection conn = new Connection();
        conn.open();   connectionList.add(conn);   activeConnections++;
        return conn;
    }

    public void releaseConnection(Connection conn) {
        if (conn != null && connectionList.remove(conn)) {
            conn.close();   activeConnections--;
        }
    }
}

DatabaseConfig dbConfig  = DatabaseConfig.getInstance();
DatabaseConfig dbConfig2 = DatabaseConfig.getInstance();
System.out.println(dbConfig == dbConfig2);      // true → same instance
```

```
ConnectionPool pool  = ConnectionPool.getInstance();

ConnectionPool pool2 = ConnectionPool.getInstance();

System.out.println(pool == pool2);              // true → same instance
```

5. Analysis

Advantages: The Singleton pattern provides a reliable mechanism for ensuring that only one instance of a critical class exists throughout the application. In this program, both DatabaseConfig and ConnectionPool are guaranteed to be instantiated only once, regardless of how many times getInstance() is called. This prevents redundant object creation and avoids conflicts caused by multiple instances holding different states. The use of synchronized on getInstance() further ensures that the pattern remains safe in multi-threaded environments, where simultaneous calls could otherwise create duplicate instances.

Disadvantage: The Singleton pattern introduces a form of global state into the application, which can make unit testing more difficult since the instance persists across test cases and cannot be easily replaced or reset. Additionally, because the constructor is private, subclassing a Singleton is not straightforward. If the application grows and requires different configurations for different environments, the rigid single-instance constraint may become a limitation that requires refactoring.

6. Conclusion

The Singleton pattern is well-suited for situations where exactly one instance of a class must coordinate actions across the system, such as managing a shared database configuration or a connection pool. In this program, DatabaseConfig ensures that all parts of the application read from the same configuration, while ConnectionPool ensures that connections are centrally managed and not duplicated across different parts of the codebase.

This approach cleanly enforces single ownership of shared resources. The client code in Application does not need to pass configuration or pool objects between methods; it simply calls getInstance() whenever access is needed, and the Singleton guarantees it always receives the same object. This is confirmed directly in the program output, where dbConfig == dbConfig2 and pool == pool2 both evaluate to true.

Overall, the Singleton pattern is a practical and controlled solution for managing shared, stateful resources in an application. It improves consistency, reduces memory overhead from redundant instantiation, and keeps resource management centralized in one place. When applied carefully and with thread-safety in mind, it serves as a fundamental and reliable building block in software architecture.